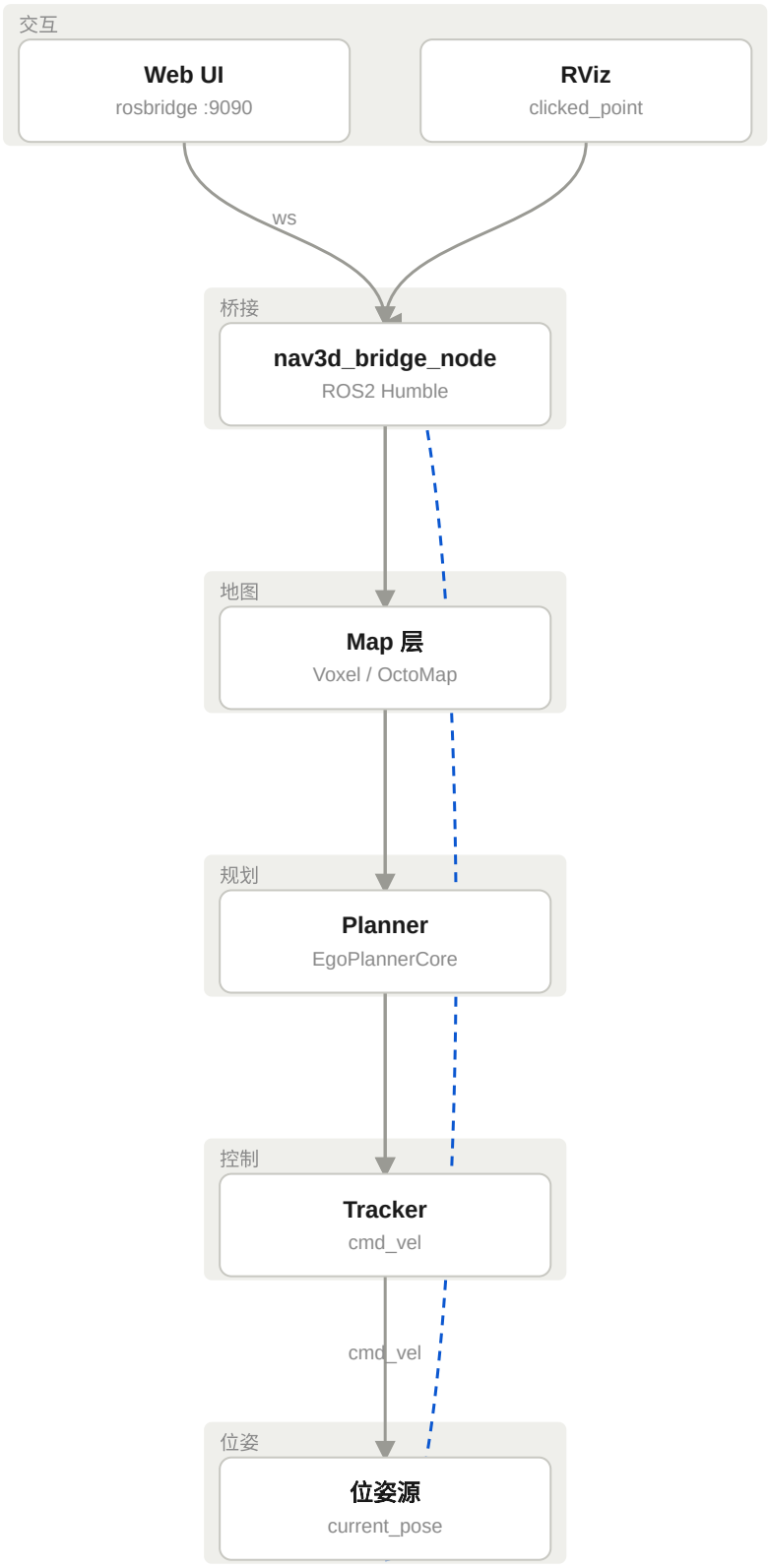
3D-Nav 算法与数据流图谱

1. 系统总览

交互发起点/目标 → bridge 建图+规划+跟踪 → /cmd_vel 驱动，/current_pose 回灌闭环



Web UI
rosbridge :9090
交互

① 浏览器连 rosbridge:9090；② 点「起点/目标」只在 UI 排队坐标；③ 点「启动」才经 WebSocket 发 /nav3d/start 与 /nav3d/goal

RViz
clicked_point
交互

① /initialpose（2D Pose Estimate）或 Publish Point 设起点；② /clicked_point 交替作为起/终点；③ 订阅 /nav3d/trajectory、

（可发 /nav3d/cancel）。Web 不跑 A*/优化，也不调 NavigateToPose。

planning_occupied_markers 看路径与碰撞图。与 Web 共用同一 bridge。

nav3d_bridge_node
ROS2 Humble
桥接

中枢节点：启动时加载 PCD 建图；收 start/goal 后 snap→planWithFallbacks；持有 active_trajectory_；定时用 current_pose 算 cmd_vel；发 status。ground 时在本文件注入 GroundSupportedSearcher。

Map 层
Voxel / OctoMap
地图

PCD 先按 resolution 体素降采样提素并去噪，再并行建成 VoxelGridMap（稠密占据）与 OctomapManager（八叉树）。运行时 LocalGrid 叠动态障碍；ground 再包 GroundTraversabilityMap。

Planner
EgoPlannerCore
规划

①网格 A* 在可通行体素上搜起点→终点折线；②折线拟合 B 样条；③ L-BFGS 平滑；④ TrajectoryChecker。成功则得到全局 active_trajectory_，跟踪器沿其 path_time 前进。

Tracker
cmd_vel
控制

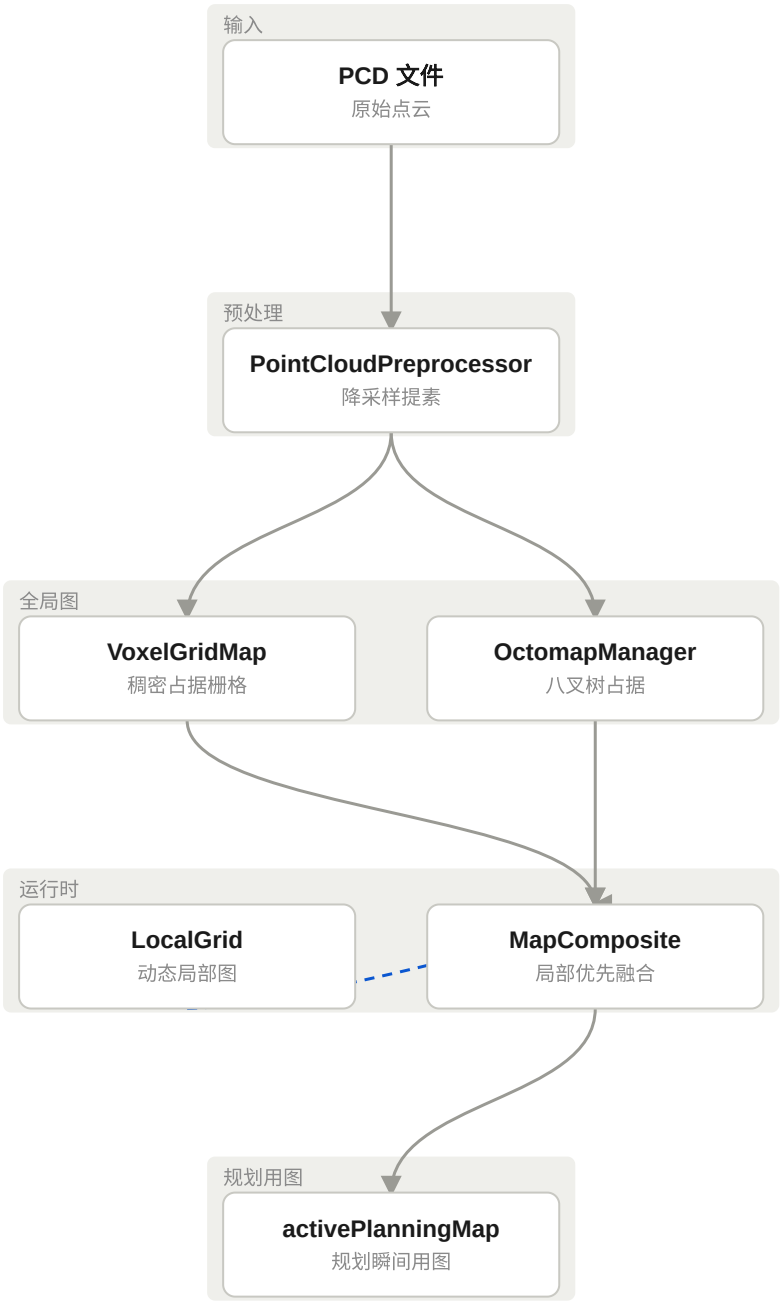
每个控制周期：用当前位姿在全局 B 样条上取前视点 → 算线速度/角速度 → 发 /cmd_vel。ground 用 Monotonic；UAV 用碰撞感知采样。不重新做全局 A*（除非 safety 触发）。

位姿源
current_pose
位姿

闭环反馈话题 /nav3d/current_pose（frame=map）。仿真：nav3d_cmd_vel_pose_sim 积分 /cmd_vel，并在 /nav3d/pose_set、/start、/initialpose 时传送。真机：换成外部定位。无 AMCL。

2. PCD → OctoMap

原始点云 → 体素降采样提素/去噪 → 并行建 Voxel 稠密图与 OctoMap 八叉树 → 运行时局部障碍叠加



PCD 文件
原始点云
输入

PcdLoader::load 读 ASCII/Binary PCD（如 building2_9.pcd）。此时还是稠密无序点，尚未成图。launch 的 pcd_path 或 /nav3d/load_pcd_path 触发加载。

PointCloudPreprocessor
降采样提素
预处理

按 preprocessor.resolution 把每个点落入三维体素格子（提素/降采样）；格子内点数不足 min_points_per_voxel 则丢弃；再按连通域剔除过小簇 min_cluster_voxels。输出洗净的稀疏点云，供建图。

VoxelGridMap
稠密占据栅格
全局图

MapBuilder::buildVoxelMapFromPointCloud 把清洗点云交给 VoxelGridMap::fromPointCloud：按 resolution 把每个点落入三维格

OctomapManager
八叉树占据
全局图

同一清洗点云 buildFromPointCloud 建成八叉树，可射线更新/序列化。仅当 planning.use_octomap_map=true 且 mode=3d 且已建好

子并写入 occupied_（同格去重）。occupied_ 有的格=障碍；边界内且不在集合中=自由。查询：isOccupied(p)=该格是否占据；isFree(p)=在界内且非占据。默认即全局规划后端（未开 use_octomap_map 时）。

时，activeGlobalPlanningMap 才用它，否则仍用 Voxel。

LocalGrid
动态局部图
运行时

订阅 /nav3d/local_pointcloud，把近期感知点写入机器人周围局部栅格。与静态全局图相对：只覆盖局部窗口，用于临时障碍。

MapComposite
局部优先融合
运行时

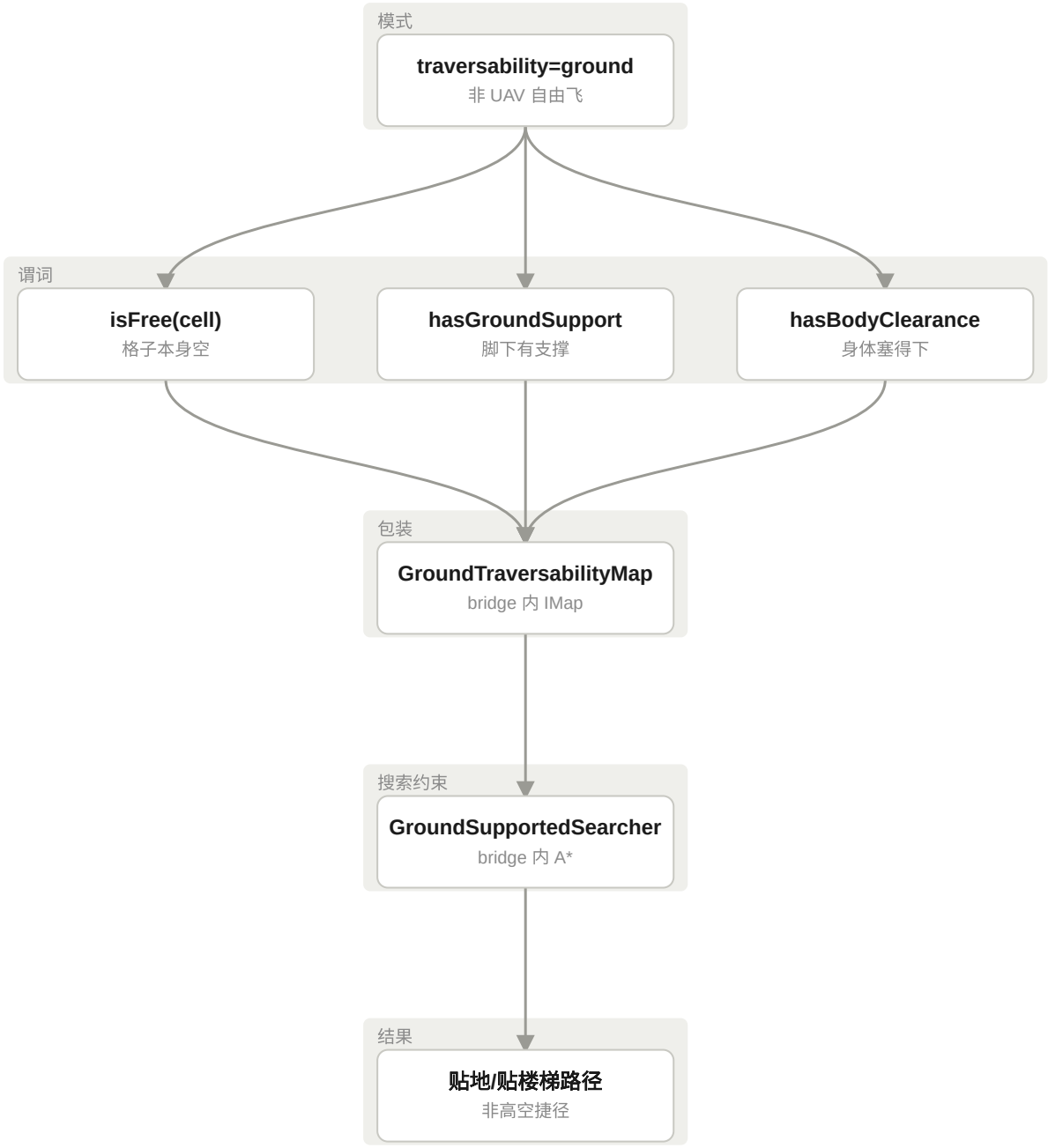
查询时先看 LocalGrid，再回落到全局 Voxel/OctoMap。可选 InflationLayer 按机器人半径膨胀障碍，保证路径留出走形空隙。

activePlanningMap
规划瞬间用图
规划用图

makePlanningMap 由内到外组装：① activeGlobalPlanningMap（Voxel 或 3D+开关下的 OctoMap）；② 有局部观测时套 MapComposite（LocalGrid 优先）；③ inflation_radius>0 时套 InflationLayer（距障≤半径也算占）；④ ground 模式最外层再套 GroundTraversabilityMap（isFree=支撑+净空）。A*/优化/跟踪查的是最外层 IMap。

3. 地面可通行

三谓词过滤可站立格子 → bridge 内 A* 只在这些格扩展 → 路径贴楼梯/楼板



traversability=ground
非 UAV 自由飞
模式

① launch：planning_traversability:=ground（UAV 模式只需自由空间）；② bridge 切换搜索器为 GroundSupportedSearcher，并把规划图包进 GroundTraversabilityMap；③ 跟踪改 Monotonic，且跳过 SafetyMonitor。

isFree(cell)
格子本身空
谓词

候选站立格不能是 occupied。2D 投影里 unknown 也当不可通行，防止穿越未观测区域。这是三谓词的第一关。

hasGroundSupport
脚下有支撑
谓词

hasBodyClearance
身体塞得下
谓词

从当前格向下在 planning.ground_support_depth_cells 深度、planning.ground_support_xy_radius_cells 水平足迹内找 occupied（踏面/楼板）。找不到 = 悬空，A* 不扩展 → 禁止高空捷径。

以格中心为球心、planning.ground_robot_radius 为半径，球体不得与占用体素相交。保证「脚能站」之外「机身也能过」楼梯狭缝。

GroundTraversabilityMap

bridge 内 IMap

包装

写在 nav3d_bridge_node.cpp：对外 IMap::isFree 改为「三谓词合取」，isOccupied 仍透传底层图。这样 EgoPlanner/A* 不用改接口，只要查 isFree 就会自动贴地。

GroundSupportedSearcher

bridge 内 A*

搜索约束

① 起终点 snap 到附近可通行格（ground_snap_radius_cells）；② 在 ground 索引上 open-set A*，邻接边 canMoveGround+采样防穿墙；③ 成功路径每个点重复 3 次（kGroundWarmStartMultiplicity）再交给 B 样条。

贴地/贴楼梯路径

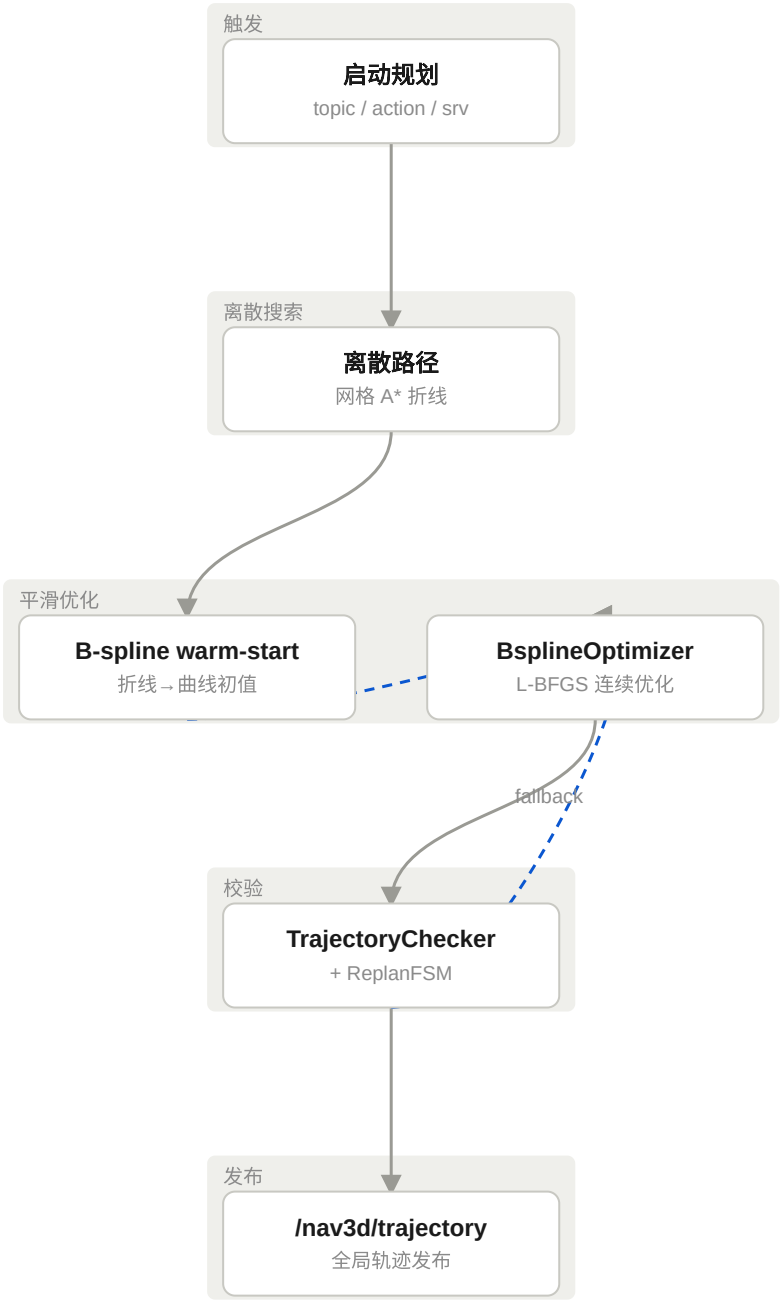
非高空捷径

结果

离散最优 = 可通行连通分量上的最短路径。楼梯踏面有支撑可走，天井中间无支撑不能跳。后续 B 样条/L-BFGS 只在该折线附近光滑，不会突然改成飞越。

4. 全局规划

网格 A* 离散折线 → B样条 warm-start → L-BFGS 平滑 → 碰撞校验；跟踪整条全局轨迹，UAV 可局部 stitch 重规划



启动规划
topic / action / srv
触发

入口三种：① Web/RViz 发 start+goal → onGoal；② NavigateToPose；③ GetPlan（只规划不跟踪）。先把起终点 snap 到可通行格，再 planWithActiveMap（一次全局：start→最终 goal）。

离散路径
网格 A* 折线
离散搜索

在规划图体素格子上跑 A*：从起点格向邻格扩展（UAV：AStar3D/JPS；Ground：仅可通行+有支撑的格），边连续采样防穿墙。输出折线航点序列（最短路径意义下的离散最优），不是最终平滑轨迹。

B-spline warm-start
折线→曲线初值
平滑优化

BsplineOptimizer
L-BFGS 连续优化
平滑优化

UniformBspline::fitThroughWaypoints 把 A* 折线拟合成均匀 B 样条，作为后续优化的初值；Ground 路径点还会重复 3 次以稳住样条形状。

在控制点上做 L-BFGS：压平滑/障碍/动力学代价。碰撞处用局部小窗口 A*（appendLocalReboundCues）生成绕障 cue（无 ESDF）。得到可跟踪的连续全局轨迹。

TrajectoryChecker
+ ReplanFSM
校验

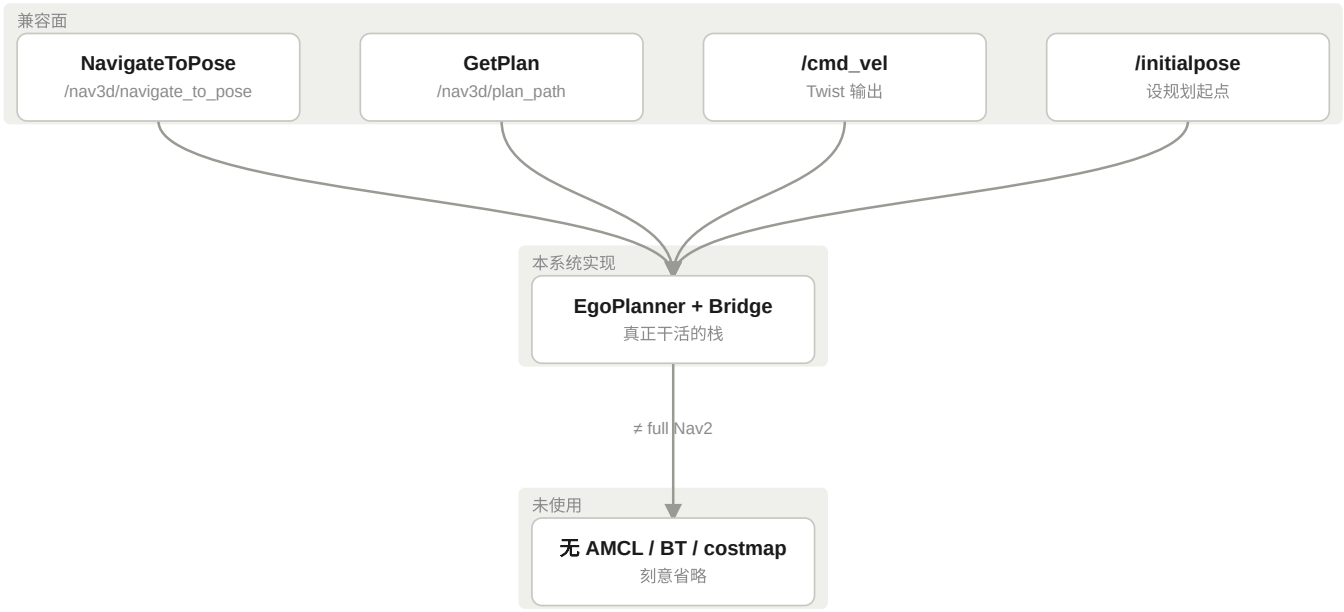
沿样条按 publish_sample_dt 密采样，查规划图是否撞障；可选动力学可行性。失败：ReplanFSM 加重碰撞权重重优化、缩短目标再搜等。耗尽 cascade 才 plan_failed。

/nav3d/trajectory
全局轨迹发布
发布

成功：active_trajectory_=B 样条，active_goal_=最终点，发布 Path+Marker，tracking_active_ 打开。之后控制环只跟这条（或 UAV stitch 更新后的版本），直到到达/取消/失败。

5. Nav2 融合

只复用接口形状（action/srv/topic）；算法在自研 bridge+EgoPlanner，不跑 Nav2 全栈



NavigateToPose
/nav3d/navigate_to_pose
兼容面

nav2_msgs/action 服务器：客户端给目标 Pose → bridge 内部规划+跟踪，并反馈 distance_remaining 等。形状像 Nav2，实现是 EgoPlanner+Tracker。Web 默认不用此 action。

GetPlan
/nav3d/plan_path
兼容面

nav_msgs/srv/GetPlan：给定起终点只算一条 Path 返回，不启动跟踪、不发 cmd_vel。适合「只要路径可视化/下游自用」。

/cmd_vel
Twist 输出
兼容面

跟踪器输出的线/角速度。名字与 Nav2 controller 习惯一致，便于接 Gazebo/真机底盘或本仓库的 pose_sim。

/initialpose
设规划起点
兼容面

RViz「2D Pose Estimate」话题。此处语义=设置/传送规划起点（并 pub /nav3d/pose_set），不做粒子滤波定位。

EgoPlanner + Bridge
真正干活的栈
本系统实现

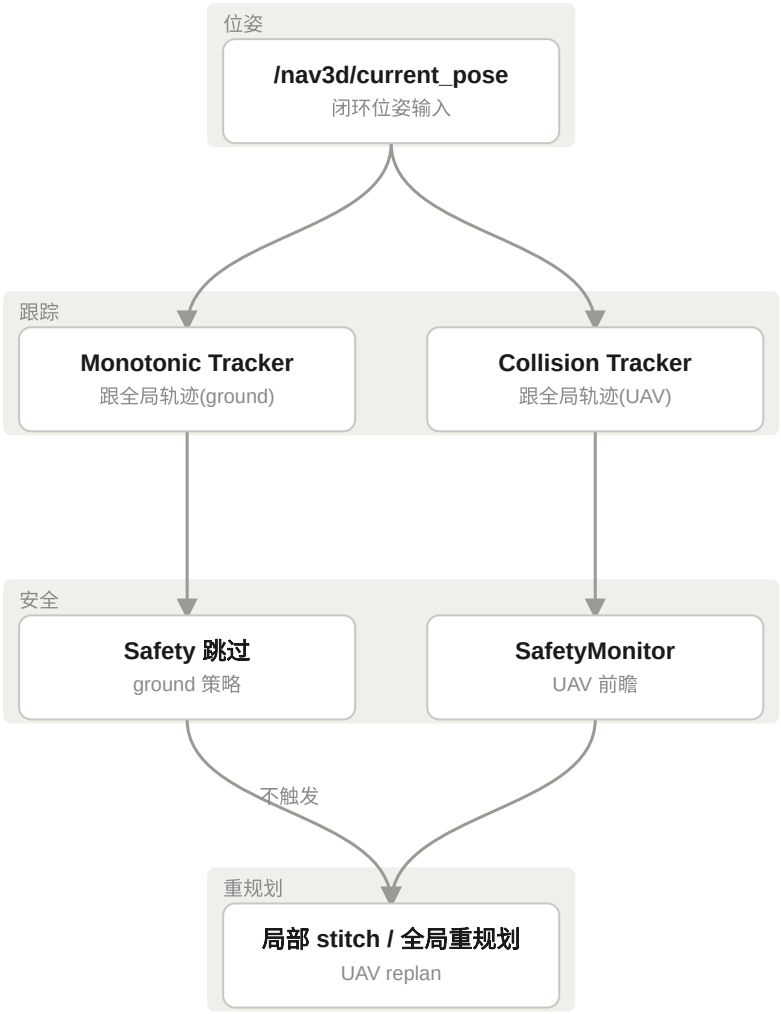
map_core / planner_core / controller_core + nav3d_bridge_node。Ground 可通行与搜索也在 bridge。没有 Nav2 lifecycle 节点图。

无 AMCL / BT / costmap
刻意省略
未使用

不跑 amcl、nav2_costmap_2d、controller_server、planner_server、behavior_tree。代价地图由自研 Voxel/OctoMap+LocalGrid 替代；定位靠外部 /nav3d/current_pose。

6. 跟踪与重规划

每周期跟全局 B 样条算 cmd_vel；Ground 跳过 safety；UAV 遇障先局部 stitch 再全局重规划



/nav3d/current_pose
闭环位姿输入
位姿

① 控制与 safety 都读它；② pose_sim：积分 /cmd_vel，并在 pose_set/start/initialpose 传送；③ 真机：外部定位直接发。frame_id 必须与 bridge 的 map 一致。

Monotonic Tracker
跟全局轨迹(ground)
跟踪

① path_time 只增不减，防止在楼梯上来回抖；② 横向偏离时在走廊半径内 snap 回轨迹；③ 仍在井内爬升时保留高度，不把 z 拉向地面；④ 近终点允许下降。输出 cmd_vel 指向前视点。

Collision Tracker
跟全局轨迹(UAV)
跟踪

同样跟 active_trajectory_：取 path_time+lookahead 目标点，查询规划图，避开占用后再生成 cmd_vel。终点判定仍对照 active_goal_。

Safety 跳过
ground 策略
安全

evaluateActiveTrajectorySafety 在 ground 直接 return：楼梯体素窄、轨迹贴边易误判，强行前瞻会反复 cancel/replan。Ground 依赖 Monotonic 几何约束保安全。

SafetyMonitor
UAV 前瞻
安全

位姿落入占用→急停；否则 TrajectoryChecker 查前方轨迹段（含 LocalGrid）。判定 ReplanNeeded 或 EmergencyStop，再交给

局部 stitch / 全局重规划
UAV replan
重规划

① 冷却 safety.replan_cooldown_sec；② 取前方 lookahead 临时目标，先 raw A* 探针；③ 可行则局部 EGO，并把新段与旧全局后半

replanFromCurrentPose。	段 stitch；④ 否则 current→final goal 全局重规划；⑤ 再失败急停并清轨迹。

六层逻辑速查

1. 与 Nav2 的关系（怎么「融合」）

① 对外只复用常见接口形状：NavigateToPose action（/nav3d/navigate_to_pose）、GetPlan 服务（/nav3d/plan_path）、/cmd_vel、/initialpose；② 收到请求后全部转进 nav3d_bridge_node → EgoPlannerCore / Tracker，不启动 Nav2 的 planner_server、controller_server、BT navigator；③ 没有 amcl / nav2_costmap_2d。因此叫「Nav2 形状兼容」，不是装了一整套 Nav2。Web 侧甚至不走 NavigateToPose，只发 /nav3d/start+/nav3d/goal。

2. PCD → 地图（怎么提素）

① PcdLoader 读入原始点云；② PointCloudPreprocessor：按 resolution 把点归入三维体素格子（降采样提素），格子内点数 < min_points_per_voxel 的丢弃，再剔除体积过小的连通簇（min_cluster_voxels）；③ 清洗后的点云并行写入 VoxelGridMap（稠密占据栅格）与 OctomapManager（八叉树）。运行时 LocalGrid 叠动态障碍，MapComposite 局部优先；可选 Inflation。规划后端：use_octomap_map 且 3D 且已建好时用 OctoMap，否则 Voxel。

3. 为何只在楼梯/地面而非高空

① launch 设 planning.traversability:=ground；② 每个候选体素必须同时满足：本身 isFree、向下 depth_cells×xy_radius 内有 occupied 支撑（踏面/楼板）、以 ground_robot_radius 为球的机体净空；③ GroundTraversabilityMap 把「可通行」写成 isFree，A*（GroundSupportedSearcher，在 bridge 内）只能扩展这些格；④ 天井/高空无支撑 → 不可扩展 → 路径只能沿楼梯连通分量上楼，不会「飞捷径」。参数均带 planning. 前缀。

4. 离散搜索 → 平滑 → 校验（全局怎么算）

① 离散搜索：在可通行体素图上做网格 A*（UAV 用 AStar3D/JPS，Ground 用 bridge 的 GroundSupportedSearcher），邻接扩展 + 边采样防穿障，得到折线路径；② 用该折线 fit B 样条作 warm-start；③ BsplineOptimizer 用 L-BFGS 平滑并压障碍/动力学代价（局部 A* rebound cue 推离，无 ESDF）；④ TrajectoryChecker 二次碰撞检查；失败则 ReplanFSM（加重碰撞权重 / 缩短目标等）再试。结果写入 active_trajectory_（整条全局 B 样条）。

5. 全局路径 vs 局部路径 vs 定位

全局路径：起点→最终目标一次规划出的 B 样条，跟踪器沿 path_time 单调前进，并发布剩余轨迹。局部路径（主要 UAV safety）：障碍触发后先在前方 lookahead 米内重规划到临时点，再与原全局轨迹后半段 stitch；A* 探针不可行或 stitch 失败则整段全局重规划到 final goal。地图上的「局部」另指 LocalGrid 动态障碍叠加。定位：外部 /nav3d/current_pose 或 pose_sim，无 SLAM/AMCL。

6. 跟踪与重规划（怎么跟、何时改线）

① 每个控制周期读 /nav3d/current_pose，在 active_trajectory_ 上取 path_time 前视点，算 /cmd_vel；② Ground 用 computeCommandMonotonic：path_time 只增、走廊 snap、爬楼梯井时保高度、接近终点可下降；且跳过 SafetyMonitor；③ UAV 用碰撞感知跟踪，并由 SafetyMonitor 前瞻；触发后 replanFromCurrentPose：冷却 → 先局部 A* 探针+stitch → 否则全局重规划 → 再失败急停。pose_sim 仅仿真时积分 cmd_vel 回灌位姿。

参数要点：planning.traversability、planning.use_octomap_map、planning.ground_support_depth_cells、planning.ground_support_xy_radius_cells、planning.ground_robot_radius、search_algorithm、safety.*